Installation and Analysis of Netrunner OS in a Virtual Environment

Course: Operating Systems and system programming

Assignment: Virtualization and OS Installation

Name: Meklit Girmaw

Student ID: 1602024

Department: Software engineering Section B

Instructor: wendimu Baye

Submisson Date: Apr 20/2018 e.c

Bahirdar university, Ethiopia

Introduction----------3
Objectives----------5
Requirements---------- 6
Hardware Requirements---------- 6
Software Requirements---------- 6
Installation steps---------- 7
Issues (Problems Faced)----------15
Issues Faced During Installation---------- 15
Solutions Applied---------- 15
Filesystem Support (Real-World Understanding)---------- 17
Advantages and Disadvantages----------18
Advantages---------- 18
Disadvantage---------- 18
Virtualization (What, Why, How)----------20
What is Virtualization?---------- 20
Why Virtualization is Used---------- 20
How Virtualization Works----------20
UNIX Standardization---------- 21
What is UNIX Standardization?----------21
Key Organizations Responsible for UNIX Standardization---------- 21
POSIX Standard---------- 21
Single UNIX Specification (SUS)---------- 22
POSIX.1 vs POSIX.2---------- 22
Extensions: POSIX.1b and POSIX.1c----------22
Role of The Open Group---------- 23
UNIX Compliance and Challenges---------- 23
Impact of Standardization on Software Development----------23
Linux, BSD, and SUS Compliance---------- 23
config.log and System Configuration----------24
Evolution of UNIX Standardization---------- 24
Conclusion of UNIX Standardization---------- 24
Conclusion---------- 25
Future Outlook / Recommendation----------26

Introduction

Virtualization has become a core technology in modern computing, enabling users to run multiple operating systems on a single physical machine. Instead of installing different systems directly on hardware, virtualization tools such as Oracle VM VirtualBox and VMware Workstation simulate hardware environments, allowing operating systems to run independently in isolated environments called virtual machines.

The operating system used in this assignment, Netrunner OS, is a Debian-based Linux distribution that uses the KDE Plasma desktop environment. It is designed to provide a balance between user-friendly design and advanced system control, making it suitable for both beginners and developers.

Background

Linux systems are heavily influenced by UNIX design principles such as modularity, multi-user capability, and strong permission control. Over time, Linux distributions like Netrunner have evolved to provide not only server-side reliability but also modern desktop usability.

From a practical perspective, working with Linux distributions gives exposure to system-level operations such as process management, file permissions, package management, and shell usage. These are essential skills in areas like backend development, cybersecurity, and DevOps.

Motivation

As a user already familiar with Linux environments such as Cinnamon-based systems, the transition into another Linux distribution like Netrunner provides a deeper understanding of how different desktop environments and system configurations operate on the same Linux kernel.

Using virtualization allows safe experimentation without risking the stability of the main system. For example:

- Testing installation steps without affecting the host OS
- Exploring system configurations freely
- Simulating real-world system administration scenarios

Practical Insight

Compared to other operating systems, Linux-based systems are considered more secure and stable due to:

- Strong user permission model (non-root default usage)

- Open-source nature (transparent and auditable code)
- Lower exposure to common malware
- Frequent updates and community-driven patches

Additionally, distributions like Cinnamon-based systems are known for simplicity and ease of use:

- Clean desktop interface
- Minimal system resource usage
- Easy navigation and configuration

This makes Linux suitable for both beginners and advanced users who want control without complexity.

Objectives

The main objective of this assignment is to gain practical and theoretical understanding of operating system installation within a virtualized environment.

Specific Objectives:

- To install Netrunner OS using a virtualization tool
- To understand how virtual machines simulate physical hardware
- To explore Linux system structure, including directories and file management
- To analyze filesystem types and their use cases in real-world scenarios
- To identify and troubleshoot issues during installation
- To apply basic shell commands for system navigation and problem-solving
- To compare different Linux environments (e.g., KDE vs Cinnamon)
- To evaluate system performance inside a virtual machine

Requirements

Hardware Requirements

The performance of a virtual machine depends heavily on the host system resources. Since virtualization shares hardware between the host and guest systems, proper allocation is important.

- **Processor:** Dual-core CPU (required for multitasking between host and VM)
- **RAM:** 4GB
    - 2GB allocated to VM
    - Remaining used by host system
- **Storage:** At least 25GB free space
- **Virtualization Support:** Must be enabled in BIOS (AMD-V in my system)

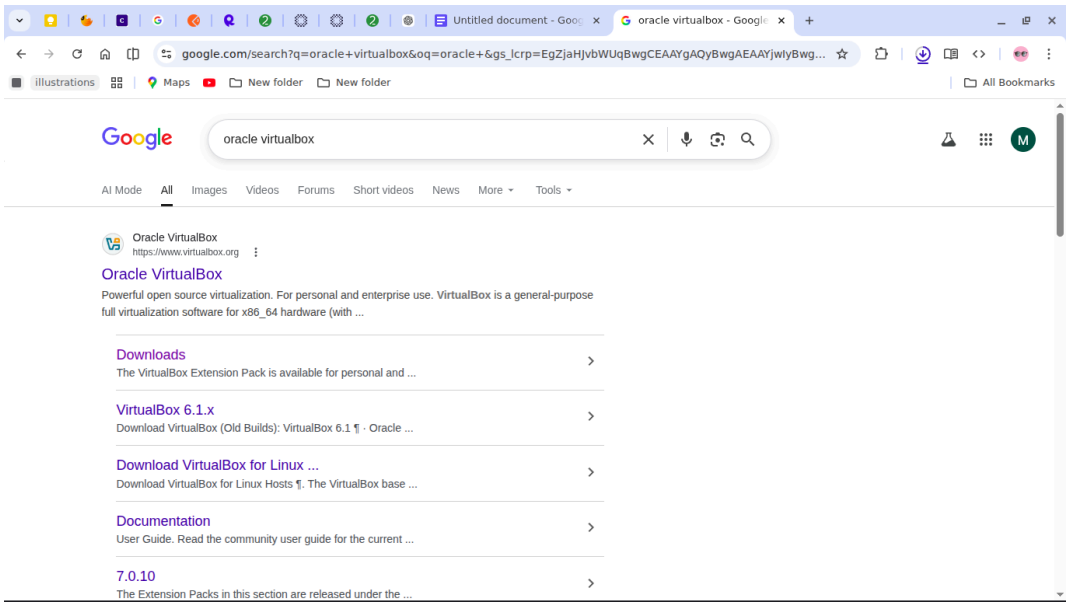
Software Requirements

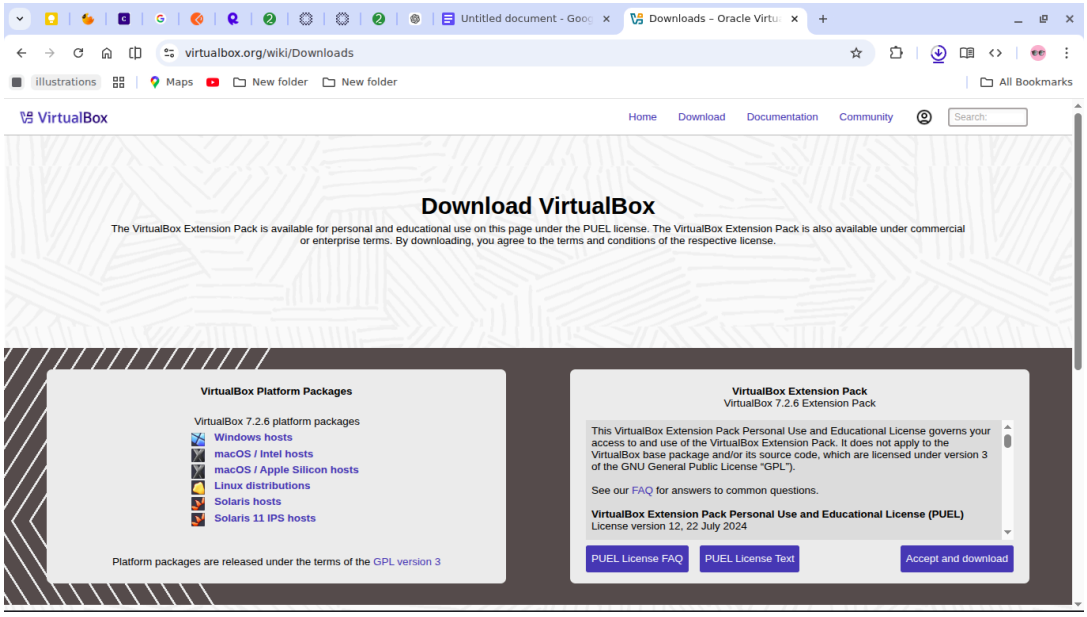
- Oracle VM VirtualBox - used to create and manage virtual machines
- Netrunner OS ISO file - installation image
- Host Operating System- Linux Mint Cinnamon

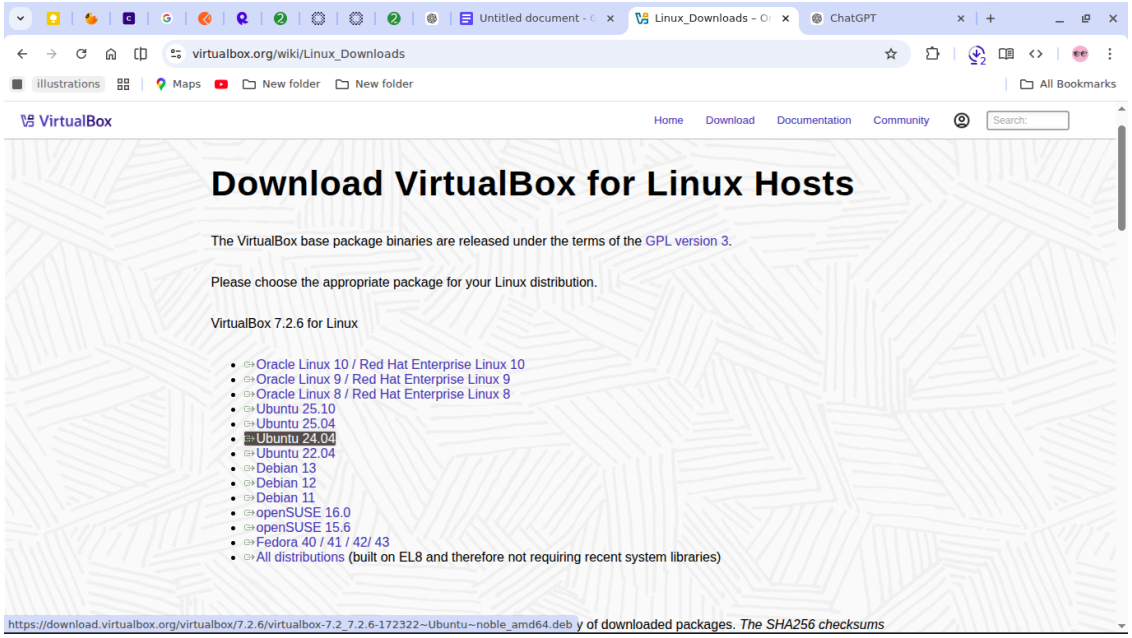
Installation steps

Step 1: Install VirtualBox

Download and install Oracle VM VirtualBox from the official website. The pictures below is going to show you how i install virtual box that is compatible with my device and OS.

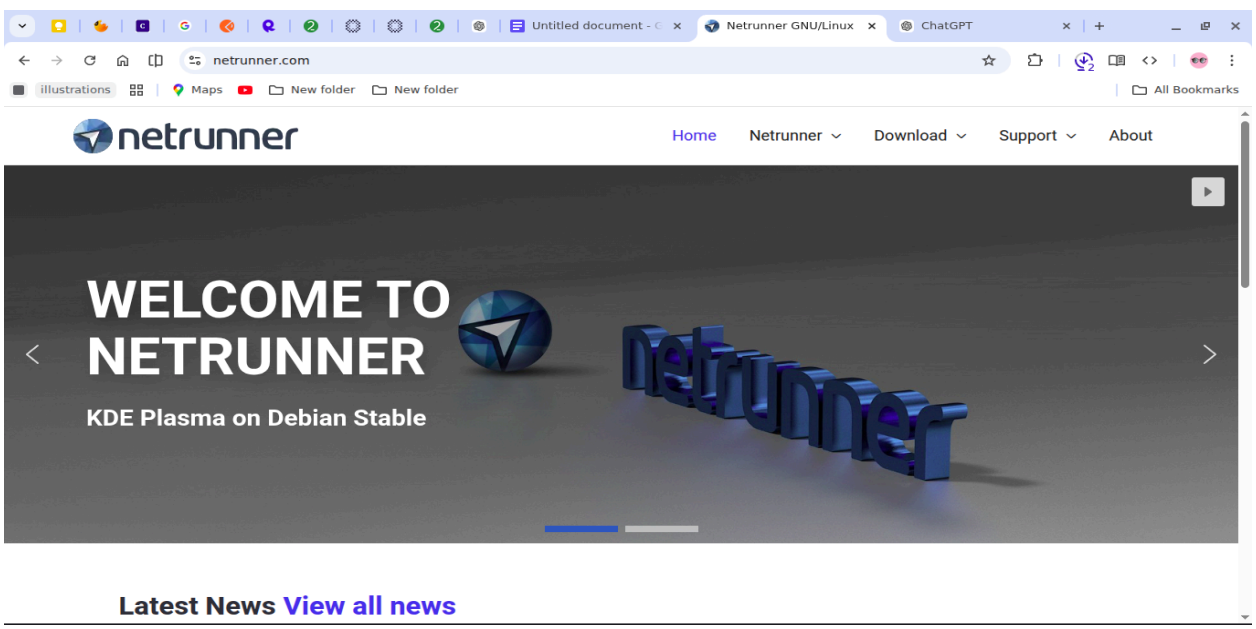


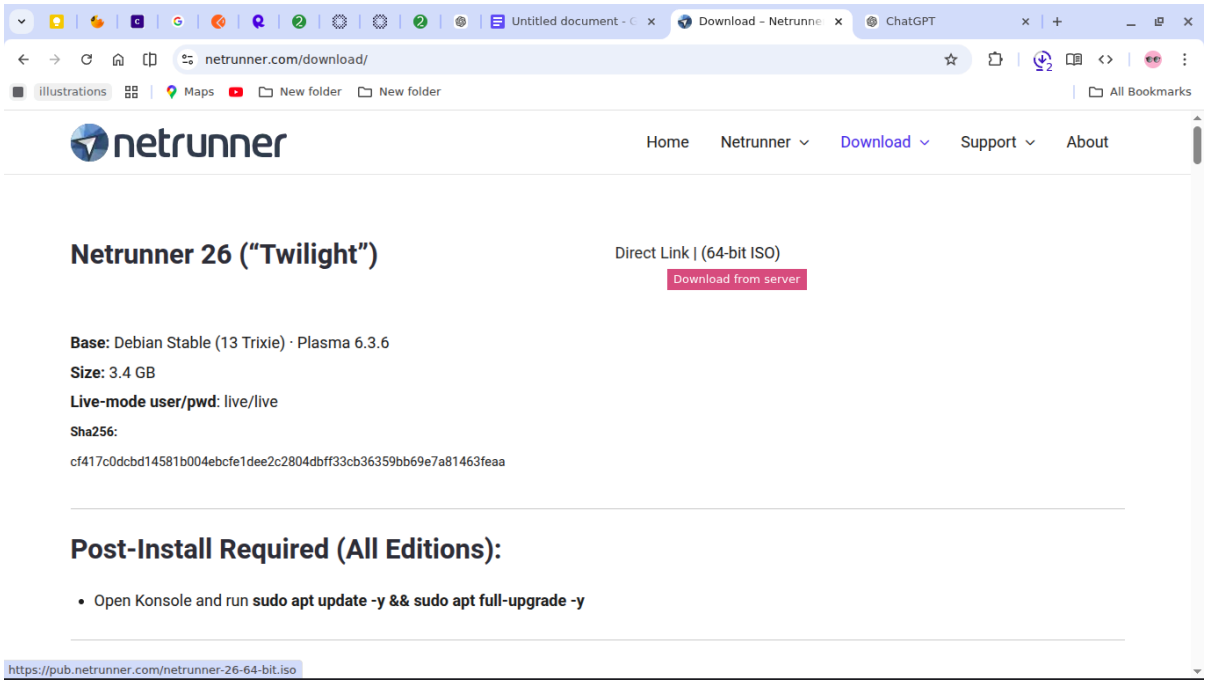


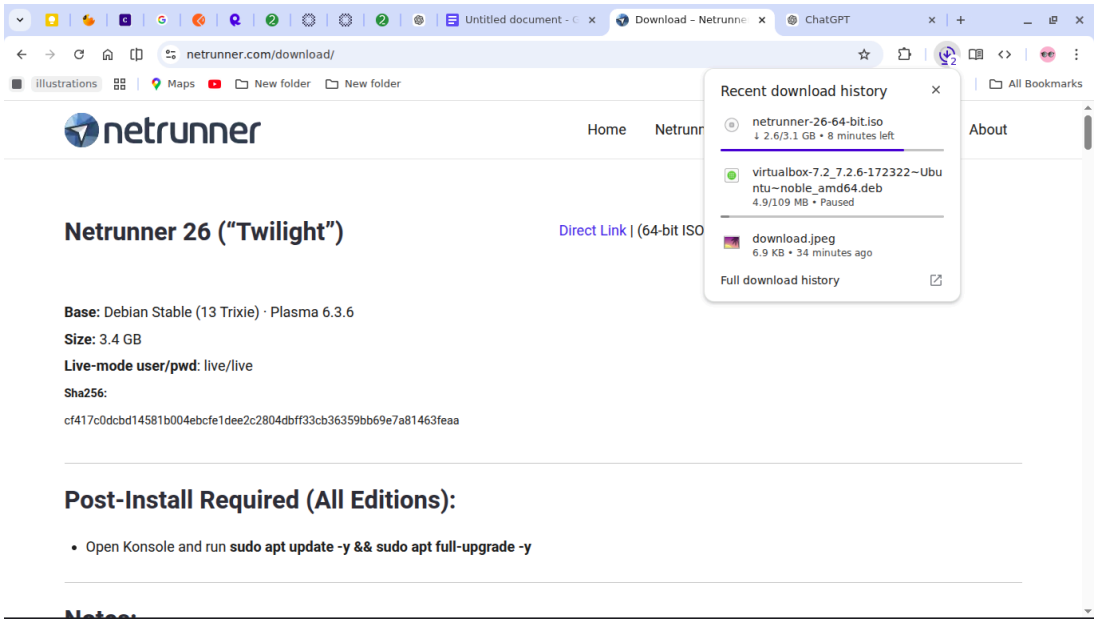


Step 2: Download Netrunner OS

Obtain the ISO file of Netrunner OS from the official source. The pictures below is going to show you how i install the latest iso file of the OS from their official site.







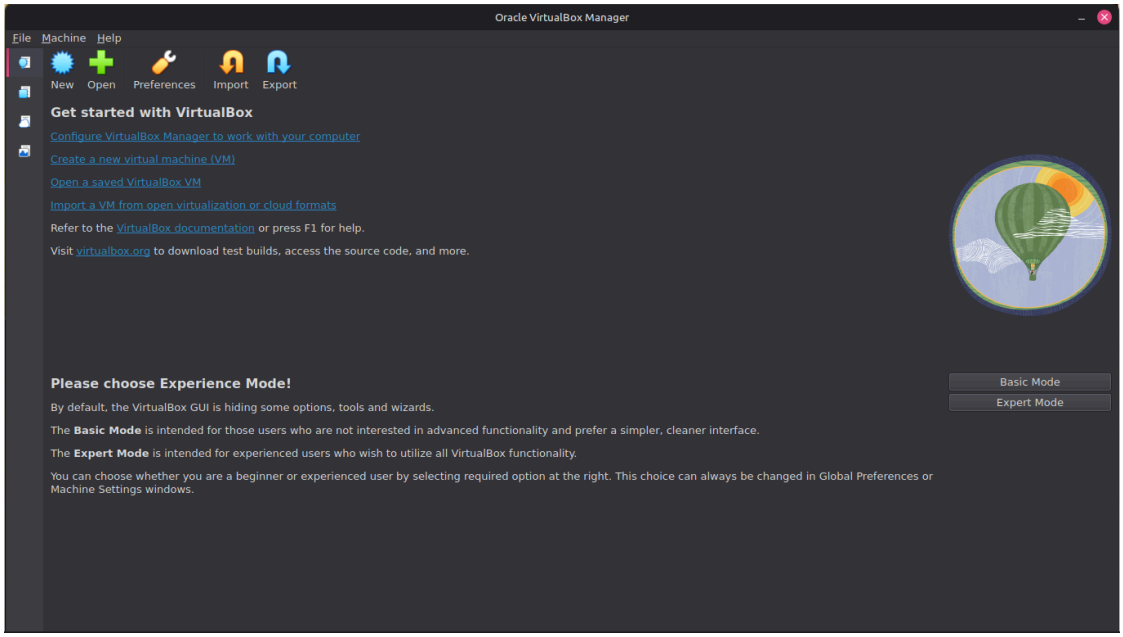
Step 3: Create Virtual Machine

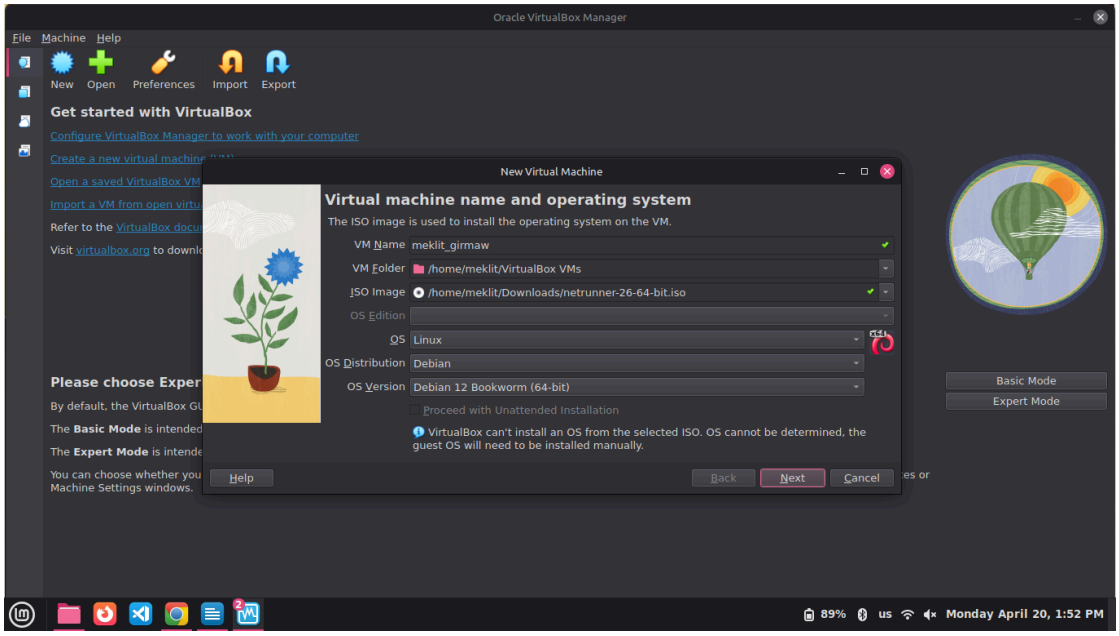
Open VirtualBox

Click “New”

Enter:

- Name: meklit_girmaw
- Type: Linux
- Version: Debian (64-bit)



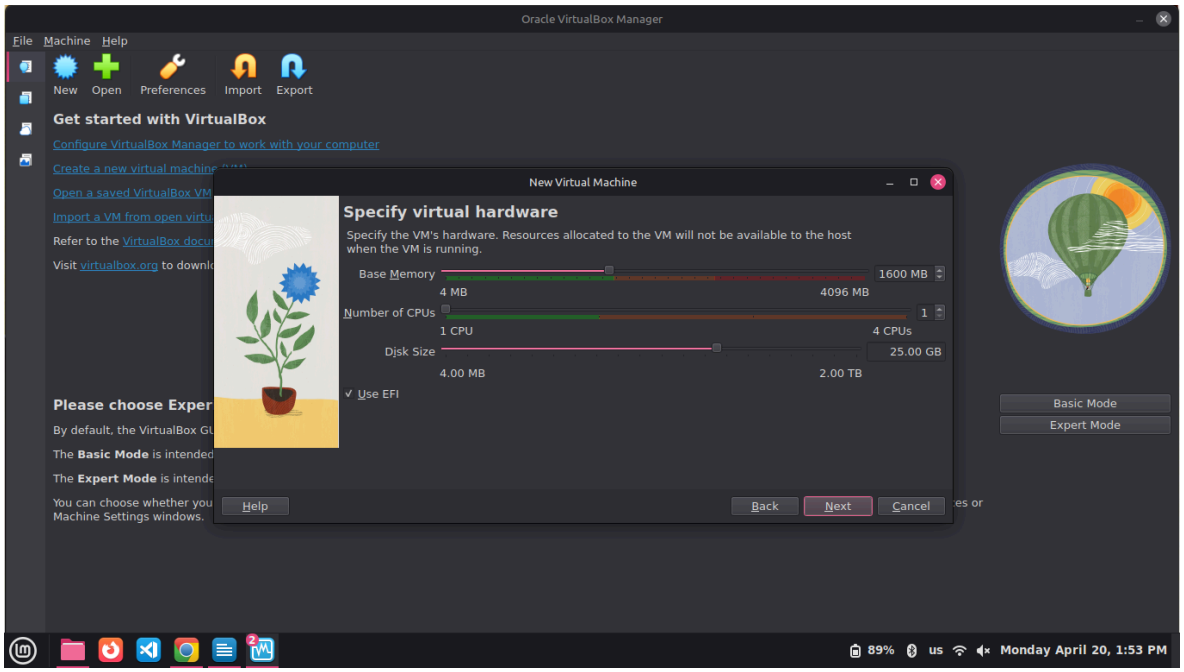


Step 4: Allocate Resources

- RAM: 2GB(Allocating too much RAM may slow down the host system, while too little can affect VM performance.)
- CPU: 1 core

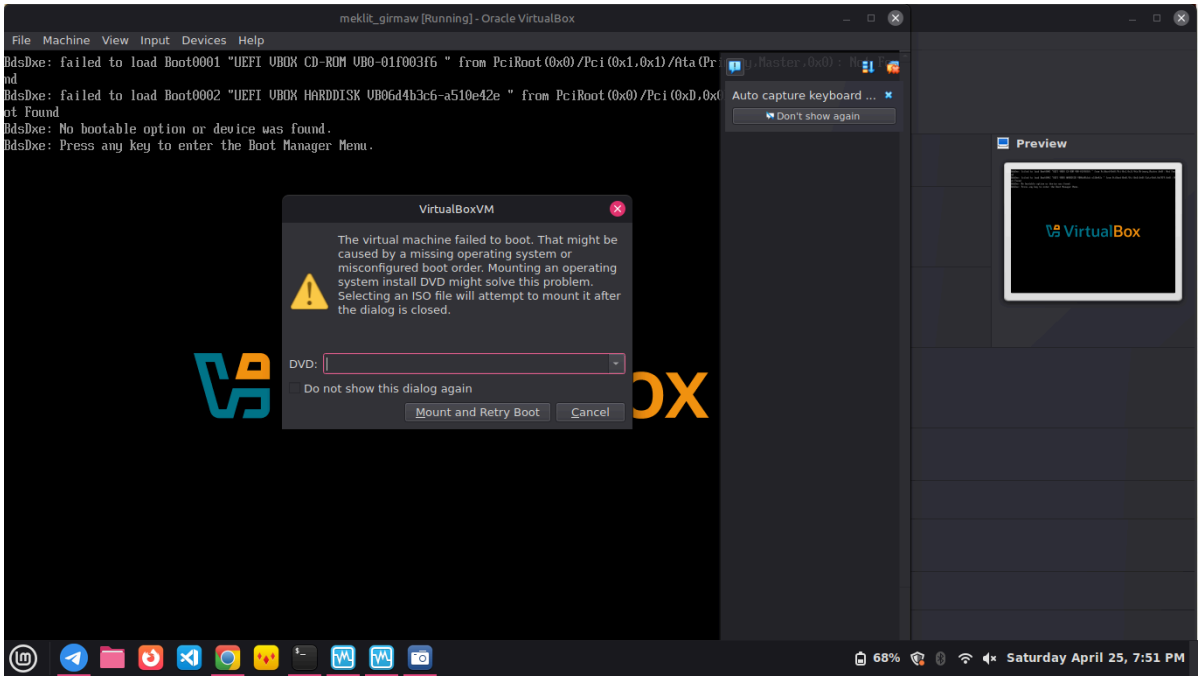
Step 5: Create Virtual Disk

- Disk Type: VDI
- Storage: Dynamically allocated
- Size: 25GB



Step 6: Attach ISO File

- Go to VM settings
- Select Storage
- Attach downloaded ISO file

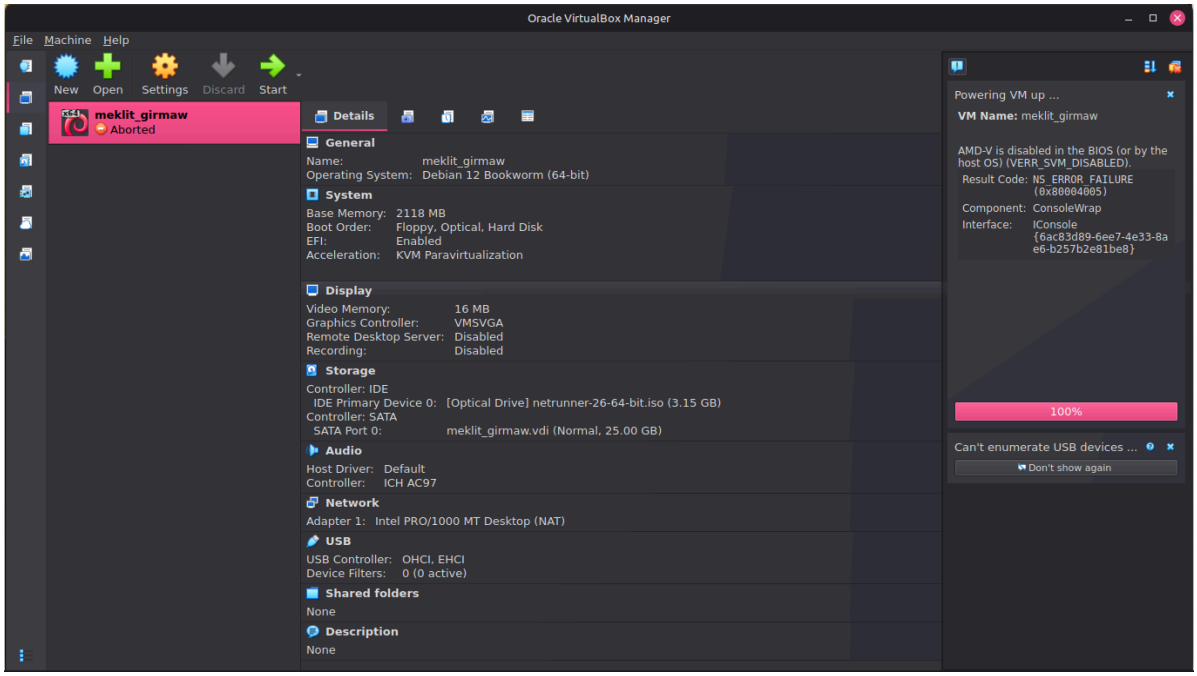


Step 7: Start Installation

- Start VM
- Follow on-screen installation steps:
    - Language selection
    - Time zone
    - Keyboard layout

Step 8: User Setup

- Username: meklit_girmaw
- Password: i assign my password

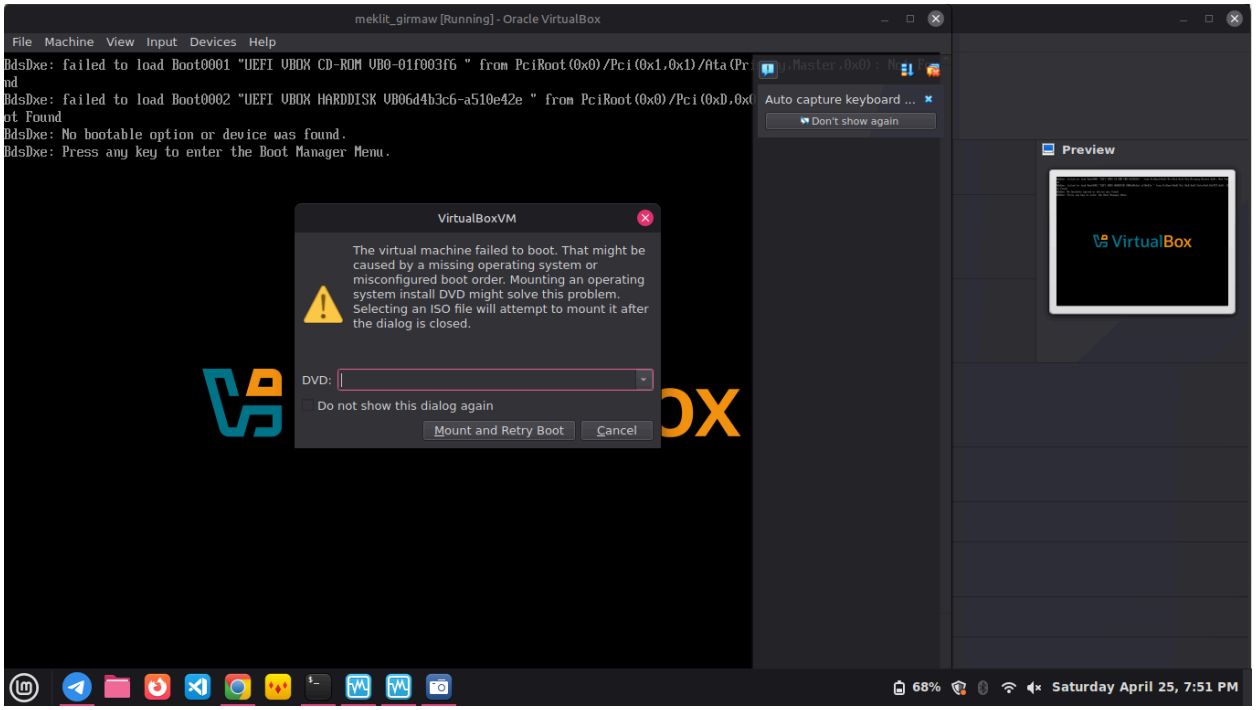


Step 9: Disk Partitioning

- Select “Use entire disk”
- Automatic partitioning

Step 10: Complete Installation

- Finish setup
- Restart VM
- Remove ISO file
- Login to system


meklit_girmaw [Running] - Oracle VirtualBox
File Machine View Input Devices Help
BdsDxe: failed to load Boot0001 "UEFI VBOX CD-ROM VB0-01f003f6 " from PciRoot(0x0)/Pci(0x1,0x1)/Ata(Pri
nd
BdsDxe: failed to load Boot0002 "UEFI VBOX HARDDISK VB06d4b3c6-a510e42e " from PciRoot(0x0)/Pci(0xD,0x0
ot Found
BdsDxe: No bootable option or device was found.
BdsDxe: Press any key to enter the Boot Manager Menu.
Auto capture keyboard ...
Don't show again
Preview
VirtualBox
VirtualBoxVM
The virtual machine failed to boot. That might be caused by a missing operating system or misconfigured boot order. Mounting an operating system install DVD might solve this problem. Selecting an ISO file will attempt to mount it after the dialog is closed.
DVD:
Do not show this dialog again
Mount and Retry Boot
Cancel
68%
Saturday April 25, 7:51 PM

Issues (Problems Faced)

Issues Faced During Installation

During the process of installing Netrunner OS inside Oracle VM VirtualBox, several practical issues appeared. These issues are common when working with virtual machines, especially on systems with limited hardware resources.

One of the first challenges was system performance. The virtual machine felt slow and sometimes lagged during booting and desktop loading. This was mainly because the host machine has limited RAM and CPU resources, and both the host system and virtual machine were sharing them.

Another issue occurred when the system failed to start properly due to virtualization being disabled in BIOS. In some cases, the VM would not even launch, showing hardware-related errors.

A small but confusing problem happened after installation: the system kept booting into the installer again instead of the desktop. This was caused by leaving the ISO file attached to the virtual drive.

There were also minor usability issues such as low screen resolution and keyboard layout mismatches, which sometimes caused incorrect password entry during login.

Overall, these issues reflect real-world challenges faced when working with virtual environments rather than physical machines.

Solutions Applied

Each problem faced during installation had a practical solution, and fixing them helped improve both understanding and system performance.

When the system was running slowly, the solution was to adjust resource allocation. Reducing background usage on the host system and properly assigning RAM and CPU to the virtual machine significantly improved performance. Enabling 3D acceleration in VirtualBox also helped smooth the interface.

For the BIOS virtualization issue, the solution was straightforward: restarting the computer and enabling Intel VT-x or AMD-V support in BIOS settings. Once enabled, the virtual machine started functioning normally.

The repeated booting into installation mode was fixed by simply removing the ISO file after installation. This ensured that the system booted directly into the installed OS instead of the installer.

Display issues were improved by installing VirtualBox Guest Additions, which helped the system automatically adjust screen resolution and improve graphics integration.

Keyboard issues were resolved by checking and matching the correct layout during login, ensuring accurate input.

These fixes are similar to real system administration tasks, where identifying and solving configuration problems is a key skill.

Troubleshooting virtual machines requires understanding both hardware constraints and virtualization software configuration, which is similar to real-world system administration tasks.

Filesystem Support (Real-World Understanding)

During this installation, the filesystem used by Netrunner OS plays an important role in how data is stored and managed.

The default filesystem used is ext4, which is widely used in Linux systems. It is known for its stability, reliability, and strong performance. One of its most important features is journaling, which helps recover data in case of unexpected shutdowns.

While ext4 is the primary choice, Linux also supports other filesystems depending on the use case.

For example, FAT32 is often used for USB drives because it is compatible with almost all operating systems, but it has a limitation of not supporting files larger than 4GB.

Another modern option is exFAT, which removes that limitation and is commonly used for external storage devices that need cross-platform compatibility.

Windows systems usually use NTFS, which is also supported in Linux, especially in dual-boot environments.

Advanced Linux users may use Btrfs, which provides features like snapshots and compression, making it useful for system recovery and backups.

In practice, ext4 remains the most balanced choice for Linux systems because it combines speed, stability, and reliability.

Advantages and Disadvantages

Using Netrunner OS gives a clear understanding of both the strengths and limitations of Linux-based systems.

Advantages

- Free and open-source: Users can access, study, modify, and improve the system without licensing costs.
- Secure system design: Strong permission model and user privilege separation reduce malware and unauthorized access risks.
- Stable performance: Systems can run for long periods without frequent crashes or slowdowns.
- Lightweight compared to Windows: Efficient resource usage allows smooth performance even on older or low-spec hardware.
- Strong community support: Large global community provides updates, fixes, documentation, and troubleshooting help.
- Highly customizable: Users can modify the desktop environment, system behavior, and tools to fit specific workflows.

Disadvantage

Linux also has some disadvantages:

- Limited software availability: Some commercial applications such as Adobe software and Microsoft Office are not natively supported.
- Learning curve for beginners: New users may find it challenging, especially when using terminal commands and system configuration.
- Hardware driver issues: Certain devices may have incomplete or inconsistent driver support depending on the manufacturer.
- Gaming limitations: Game compatibility and performance are generally better on Windows, although this has improved with tools like Proton.

Despite these limitations, Linux remains widely used in professional environments such as servers, cloud computing, cybersecurity, and software development due to its stability, security, and flexibility.

Virtualization (What, Why, How)

What is Virtualization?

Virtualization is the process of creating a virtual version of a computer system using software. Instead of using real physical hardware, a virtual environment simulates CPU, memory, storage, and network components.

In this assignment, Oracle VM VirtualBox was used to run Netrunner OS inside a virtual machine.

Why Virtualization is Used

Virtualization is widely used in modern computing because:

- It allows multiple operating systems to run on one machine
- It reduces hardware cost
- It provides a safe testing environment
- It isolates systems to prevent damage to the host OS
- It is widely used in cloud computing and DevOps environments

How Virtualization Works

The process works through a software layer called a **hypervisor**.

- The hypervisor sits between hardware and operating systems
- It allocates CPU, RAM, and storage to virtual machines
- Each virtual machine behaves like an independent computer

This is why multiple OS environments can run simultaneously on one physical device.

UNIX Standardization

What is UNIX Standardization?

UNIX standardization is the process of defining common rules and interfaces so that all UNIX-like systems behave consistently.

The main significance is:

- It guarantees **portability of software** across systems
- It reduces dependency on specific hardware or vendor implementations
- It ensures long-term stability of system interfaces
- It supports the development of reusable system-level software

This ensures that software written for one system can run on another without major changes.

Key Organizations Responsible for UNIX Standardization

- IEEE - defines technical standards like POSIX
- The Open Group - manages UNIX certification
- X/Open Consortium - early standard development body

POSIX Standard

POSIX (Portable Operating System Interface) is a set of IEEE standards that defines a set of rules that UNIX-like systems must follow (how operating systems should behave, especially in terms of system calls, shell behavior, and utilities).

Importance:

- Enables **application portability** across UNIX-like systems(Ensures software portability)
- Defines a consistent **programming interface (API)**
- Ensures that system tools behave similarly across platforms(Standardizes system calls and commands)
- Forms the foundation for Linux and UNIX compatibility(Reduces compatibility issues)

Without POSIX, software written for one UNIX system would require major rewriting for another system.

Single UNIX Specification (SUS)

SUS defines a complete UNIX system standard.

Goals include:

- Uniform system behavior
- Compatibility across vendors
- Certification of UNIX systems

POSIX.1 vs POSIX.2

These two standards define different layers of system behavior:

- **POSIX.1** focuses on core system services
    - Process management
    - File and directory operations
    - System calls and APIs
- **POSIX.2** focuses on user-level tools
    - Shell commands
    - Command-line utilities
    - Scripting behavior

In simple terms, POSIX.1 defines how the system works internally, while POSIX.2 defines how users interact with it.

Extensions: POSIX.1b and POSIX.1c

- **POSIX.1b (Real-time extensions):**
    - Adds real-time scheduling
    - Supports high-priority process execution
    - Used in systems requiring time-critical performance
- **POSIX.1c (Threading support):**
    - Introduces POSIX threads (pthreads)
    - Enables multi-threaded programming
    - Improves performance in modern multi-core systems

Role of The Open Group

The Open Group plays a central role in UNIX standardization by:

- Defining the Single UNIX Specification (SUS)
- Certifying operating systems as UNIX-compliant
- Maintaining interoperability between UNIX vendors
- Ensuring consistency across UNIX implementations

Only systems that pass The Open Group certification can legally be called “UNIX.”

UNIX Compliance and Challenges

UNIX implementations ensure compliance with POSIX standards by:

- Following standardized system libraries
- Passing conformance test suites
- Maintaining compatibility layers

However, vendors face challenges such as:

- Hardware-specific differences
- Performance optimizations that conflict with standards
- Balancing innovation with strict compliance rules
- Fragmentation between UNIX-like systems

Impact of Standardization on Software Development

UNIX standardization has had a major impact on software engineering:

- Developers can write code once and run it on multiple systems
- It reduces maintenance cost for large systems
- It improves collaboration between different platforms
- It enabled the growth of portable software ecosystems

This is one of the key reasons modern infrastructure (cloud, servers, DevOps) heavily relies on UNIX-like systems.

Linux, BSD, and SUS Compliance

Systems like Linux and BSD are not fully UNIX-certified, but they are highly compatible:

- Linux follows POSIX standards closely but is not formally certified
- BSD systems are closer to traditional UNIX implementations
- Both rely on standard interfaces to maintain compatibility with UNIX software

This hybrid approach allows flexibility while maintaining portability.

config.log and System Configuration

The config.log file is generated during software compilation and configuration processes. It:

- Records system environment checks
- Logs configuration decisions
- Helps developers debug build issues
- Ensures compatibility with system standards

It indirectly supports UNIX compliance by documenting how software adapts to system differences.

Evolution of UNIX Standardization

UNIX standardization evolved in stages:

- Early UNIX (AT&T era): no formal standards
- 1980s: initial standardization efforts to unify UNIX behavior
- POSIX introduced by IEEE to formalize portability
- UNIX 95 and UNIX 03: formal certification standards
- Modern era: Linux and BSD systems adopt POSIX compatibility

Conclusion of UNIX Standardization

UNIX standardization, driven by POSIX and The Open Group, has been essential in shaping modern operating systems. It ensures portability, stability, and interoperability across different platforms. Even non-certified systems like Linux rely heavily on these standards, proving their importance in real-world computing environments.

Conclusion

This assignment helped me understand virtualization in a practical and simple way. Instead of just reading about operating systems, I was able to actually install and run one using Oracle VM VirtualBox. This made concepts like virtual machines, hardware simulation, and resource allocation much clearer.

Working with Netrunner OS also gave me hands-on experience with Linux. I learned how the system is structured, how files are managed, and how users interact with the operating system. Even the problems I faced, like slow performance, BIOS settings, and boot issues, were useful because solving them felt like real-world system administration work.

Overall, virtualization made it safe to experiment, make mistakes, and learn without affecting my main system. It showed me that learning operating systems is not just theory it becomes much easier when you actually try it yourself.

Future Outlook / Recommendation

Virtualization is becoming more important in modern tech, especially in cloud computing, DevOps, and backend systems. Learning it now builds a strong base for future tools like containers and cloud platforms.

For developers, it is a good idea to try different Linux distributions depending on your goals:

- Fedora is a strong choice for developers because it provides up-to-date software, good security, and works well with modern development tools. It is especially useful if you want to stay close to new technologies.
- Arch Linux is better for advanced users who want full control and a deeper understanding of how Linux works. It teaches a lot, but it requires more effort and time.

At the same time, other operating systems are still important for specific use cases:

- Windows is useful for software that is not supported on Linux, such as some design tools, office applications, and gaming.
- macOS is popular for mobile app development (especially iOS) and is widely used in professional environments.

To improve further, it is recommended to:

- Practice running multiple virtual machines
- Try different Linux distributions and compare them
- Learn basic networking inside virtual environments
- Explore container tools after understanding virtualization

In the future, combining virtualization skills with real development work will make it easier to build, test, and deploy applications in professional environments.